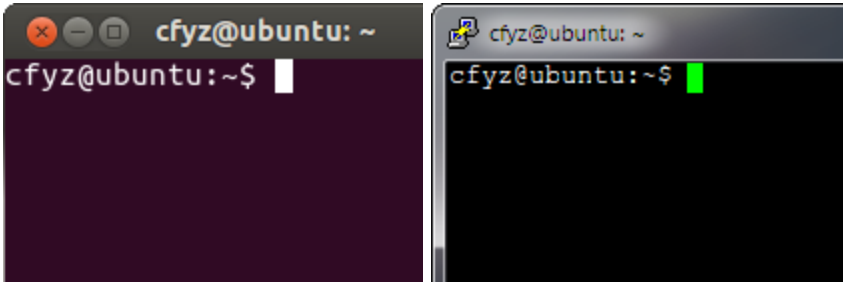
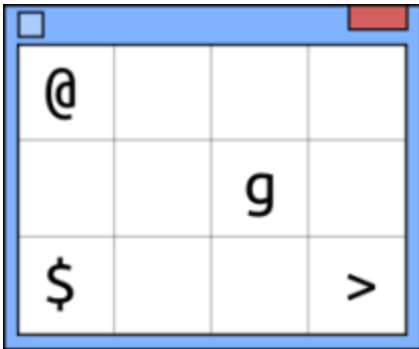


BearLibTerminal / Design Overview

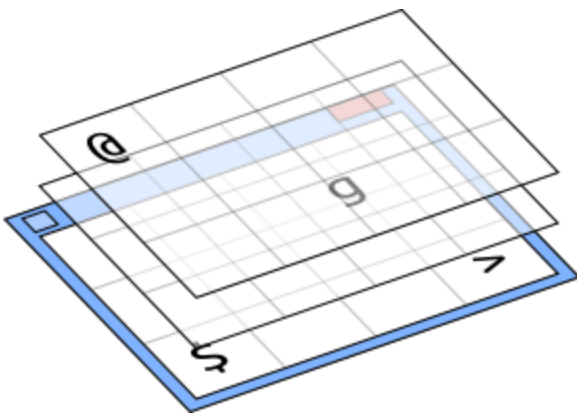
The main purpose of the BearLibTerminal library is (or at least, it was) to provide user with a window impersonating the system console, but much more customizable and geared more toward the game development. Here is what the BearLibTerminal window may look like. Spot the differences with the original ones =)



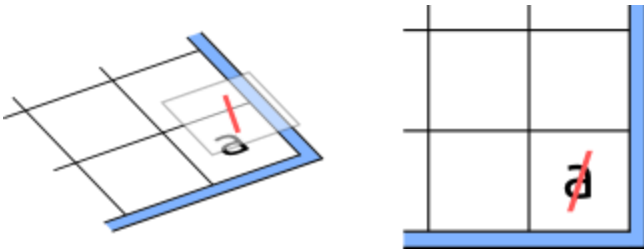
That is how it *looks*. However, the way it *works* is different from a console. You don't get a continuous in/out streams to write to and to read from. Instead, you get a grid of cells which can be addressed independently, a *scene*.



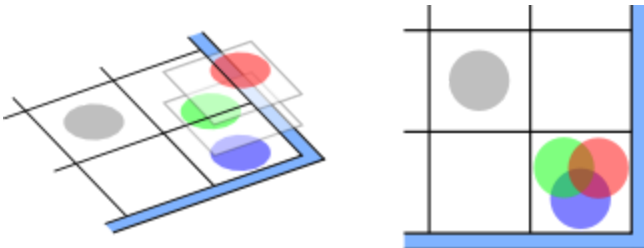
To be precise, a scene is not a single grid, but a number of them, called *layers*. For a simple game you might not need more than one layer, but sometimes they can be indispensable.



Each cell, in turn, can store a number of *tiles*. Because the library takes full advantage of alpha (transparency) channel of tile images, this effectively allows to combine several images into one, constructing them on the fly instead of preparing tiles for every combination beforehand. Note that each tile in a stack can have its own color.



Whats more, each tile in a stack can have its own offset from the default position in a cell.



Now, where are those tiles come from?

Tiles and slots

In BearLibTerminal each character code can have a tile associated with it, e. g. a tile for '@' or a tile for '#'. The range of character codes the library operates on is [basic multilingual plane](#) of Unicode. This means there are around 65k of slots for tiles.

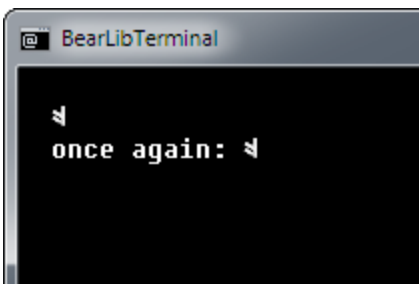
Assigning a single tile to the slot is very straightforward:

```
terminal_set("0x5E: tile.png");
```

Which effectively replaces whatever there was in the slot with a new image. In the given example it was a circumflex character. Now it can be used just like any other glyph:

```
terminal_put(2, 1, 0x5E);
terminal_print(2, 2, "once again: ^");
```

Which will produce something like this:



Note that tile size is not restricted by the library. You can have a tile as big as the window, e. g. background image for a menu. By default tile is centered in a cell but it can be easily overridden.

Bitmap tilesets

Assigning images one slot at a time is tedious and positively counterproductive. BearLibTerminal allows loading whole tilesets at once. Lets say a tileset contains four tiles, each of them is 16×16 pixels in size. It is loaded in almost the same way as a single tile:

```
terminal_set("0x1000: tileset.png, size=16x16");
```

The only difference is that you have to specify a size of a single image in the tileset bitmap. The library will slice the bitmap up into pieces and assign them to the character code points. The order BeralibTerminal iterates over them is fixed: row by row, from left to right. By default the tiles are placed into consecutive slots. In the example above there will be four tiles assigned to character points 0x1000 through 0x1003.

There is no difference between a tileset and a tile. A single tile is just a small tileset with one image.

The number specified as the starting code point identifies the tile or tileset. To remove the loaded tilesets, you just have to say that there is nothing in the slots:

```
terminal_set("0x5E: none; 0x1000: none");
```

Bitmap fonts and codepages

Now there is a bit tricky part about using bitmap tilesets as a font. BearLibTerminal uses the Unicode plane for tiles. Usually you can just write:

```
terminal_print(0, 0, "Unicode: α"); // UTF-8
```

And the library will look up the tile for greek symbol alpha in the U+03B1 slot, the number assigned by the Unicode standard. But most of the bitmap fonts usually come as a single image where all the glyphs are packed together, for example:



While the alpha symbol is 0x03B1 in Unicode, it may be placed anywhere in the image. For example, in the Dwarf Fortress font shown above, the beta symbol is in the 15th row, 1st column, which is the 224th image piece. Far from the 0x03B1. And no matter what starting code point you choose, you won't get all of the characters in their proper Unicode slots.

This is where a *codepage* is used. A codepage is a simple mapping from one character code space to another. In this case it is a mapping from indices in the font bitmap to the Unicode code points. By specifying a codepage when loading a tileset you can instruct the library to place tiles not consecutively but according to some list.

```
terminal_set("font: Nobbins.png, size=9x12, codepage=437");
```

The codes in a codepage are offsets from the starting code point. In the example above, the library reads a mapping for 224th index (which is 0x03B1 in the codepage 437), adds it to the starting point (the “font” part is just a synonym to zero) and places the tile in the right slot.

It just happens that Dwarf Fortress fonts are in codepage 437. There are a few codepages built into the BearLibTerminal, namely [437](#), [866](#), [1250](#) and [1251](#). However, you can use any codepage, e. g. custom one, by specifying a name of the codepage file:

```
terminal_set("font: font.png, size=9x12, codepage=custom.txt");
```

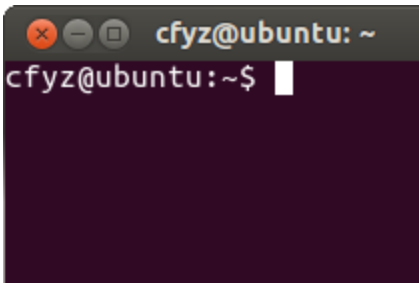
As you can see now, the main font is just a tileset loaded from starting code point “0”. There is a default built-in bitmap font generated from the [Fixedsys Excelsior](#) with a codepage matching the [WGL4](#) list, which is automatically selected upon library initialization.

TrueType tilesets

A kind of a killer feature for BearLibTerminal is how it can use TrueType (.ttf) fonts as a source for tilesets. You can set it as a main font:

```
terminal_set("font: UbuntuMono-R.ttf, size=12");
```

And the library will take care of rasterizing the tiles and placing them into correct slots. An example was already shown at the very top of the page:



The way TrueType tilesets work is almost the same as bitmap tilesets. You can load several of them to a different starting code points. However, there are some differences.

The simplest one is the way TrueType tilesets handle the “size” parameter. Here the option means the size to which the vector glyphs will be rasterized to, and can come in two forms:

- `size=12`: it specifies an average height of a lowercase letter, in pixels. Be wary that depending to the font face, the actual size of a tile may vary wildly.
- `size=8x16`: it specifies a size of a tile directly. The library will try and use the biggest font size fitting this size.

The other difference is that the codepage meaning is pretty much inverted. Nowadays TrueType fonts come with a codepage built in, so the BearLibTerminal does not need an assistance to figure out where each tile goes. This also means loading a TrueType tileset to some arbitrary starting code point will mess things up by introducing an offset to the mapping. Therefore, codepage for a TrueType tileset describes a reverse mapping: from Unicode to relative indices. This allows to load a select number of “characters” from a TrueType font to a number of consecutive slots. For example:

```
terminal_set("0xE000: fontawesome.ttf, size=16x16, codepage=fontawesome.txt");
```

with the following codepage:

```
0xF00C, 0xF062, 0xF001, 0xF0E7, 0xF013, 0xF043
```

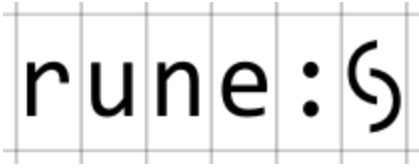
will place exactly six ideograms from the [FontAwesome](#) font to slots 0xE000 through 0xE005.

Tile alignment

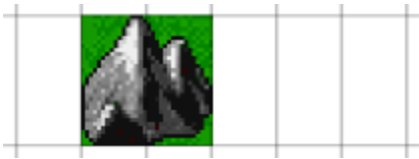
As noted somewhere above, tiles have alignment property which affects how they are drawn relative to the position of a cell. All tiles in a tileset share the same alignment:

```
terminal_set("0xE000: tileset.png, size=10x20, align=center");
```

The default tile alignment is “center” and can be omitted when loading a tileset. This alignment is generally useful for character tiles because it places them in a usual way and allows for some size tolerance without visually breaking a line:



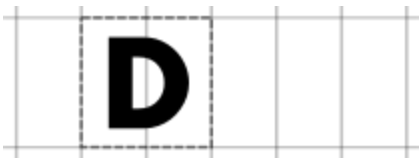
The other available tile alignments are: “top-left”, “bottom-left”, “top-right” and “bottom-right”. The top-left alignment is useful for tiles which are clearly bigger than one character cell, e. g. terrain tiles or monster pictures in a book:



The tile alignment can be further customized by specifying the size of an alignment region. The default size is 1×1 cell and can be overridden by the “spacing” attribute:

```
terminal_set("0xE000: tileset.png, size=20x20, align=center, spacing=2x1");
```

It can be useful for cases like multi-cell monsters or initial letters:

[Show pagesource](#)[Old revisions](#)[Media Manager](#)[Login](#)[Sitemap](#)[Back to top](#)